

# 2011 年参考答案

## 一、选择题

1. B 有向图时:  $n(n-1)$  条边, 两结点之间双向指向;  
无向图时:  $n(n-1)/2$  条边, 单向指向。  
注意: 可以画一个草图, 有时候更为方便快捷, 比如涉及: 树, 图, 栈, 队列等数据结构时, 可快速解题。
  2. A 可联想生活中的排队现象, 先进先出的原则;  
栈操作的原则是: 先进后出。  
注意: 涉及简单知识点时, 可以直接背诵记忆。
  3. B 栈顶指针  $\text{top}$  指向栈顶元素, 故先进行操作, 后指针-1。
  4. A
  5. D 顺序表的显著特点是: 随机存取。
  6. A 链表的显著特点是: 方便插入与删除, 由于查找元素时, 需从头开始查找, 故需要花费大量时间。
  7. B
  8. A 删除时, 需要移动  $n-i$  个元素;  
插入时, 需要移动  $n-i+1$  个元素。
  9. C 考察栈: 先进后出。
  10. C
  11. B
  12. C
  13. A 最短时间: 从源点到汇点的最长路径的长度。
  14. C 物理地址与逻辑地址相同时, 顺序存储结构。
  15. B
  16. D 有向图中: 出度之和 = 入度之和 = 边数。
  17. D
  18. D 判断有向图是否有回路, 常用: 拓扑排序, 深度优先遍历算法。
  19. A
  20. A
- 注意: 选择题、填空题一般考察比较简单, 基本上所涉及的知识点, 那本重要知识点总结书上都可查。

## 二、填空题

1. 归并排序  $O(n \log n)$
2. 计算简单, 散列函数分布均匀
3. 一端, 先进后出
4. 顺序或链式,  $(n+1)/2$
5. 任意结点出发可遍历整个链表
6. 时间, 空间
7.  $n-1$ ,  $n(n-1)/2$  可以举一个简单的序列, 理解记忆
8.  $2n-1$  新增  $n-1$  个结点, 共  $n+n-1$
9.  $n, n$



### 三、判断题

1. ×
2. ✓
3. × 可能是非连通图
4. ✓  $n \times n$  大小
5. ×
6. ×
7. × 构造树的组合为：先序与中序，中序和后序
8. ✓
9. × 最大比较次数为高度  $h$
10. ✓
11. × 常考点
12. ✓
13. × 顶点的度包括出度和入度，而顶点链表为空，是其中一种情况。
14. × 图内出现环
15. ✓

### 四、应用题

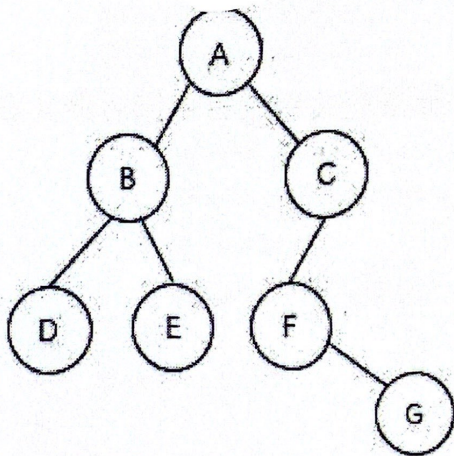
1. (1) 区别：数据的逻辑结构是一个数学模型，数据的物理结构是数据的逻辑结构在计算机内部的存储方式。

(2) 联系：一种逻辑结构可以用多种存储结构来存储；一种存储结构可以用来存多种逻辑结构。

2. (1)  $a \rightarrow b \rightarrow c \rightarrow e \rightarrow d$

(2)  $a \rightarrow d \rightarrow b \rightarrow c \rightarrow e$

3.



4.

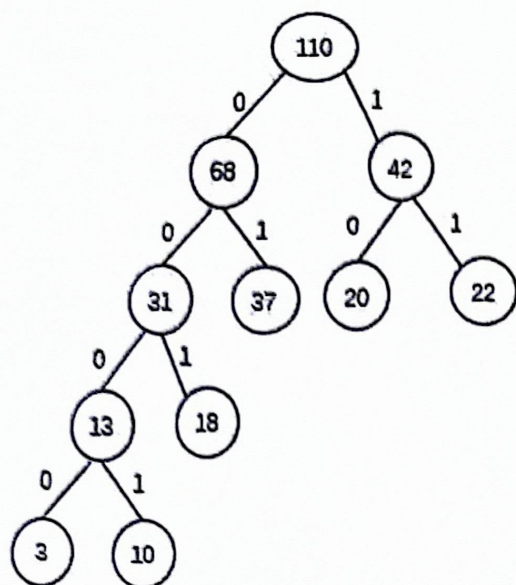
$t = \text{'These are books'}$

$v = \text{'nmn'}$

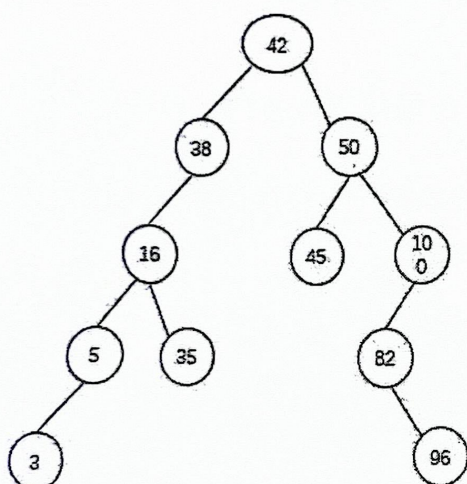


u='mcmcmc'

5.左 0 右 1 得哈夫曼编码

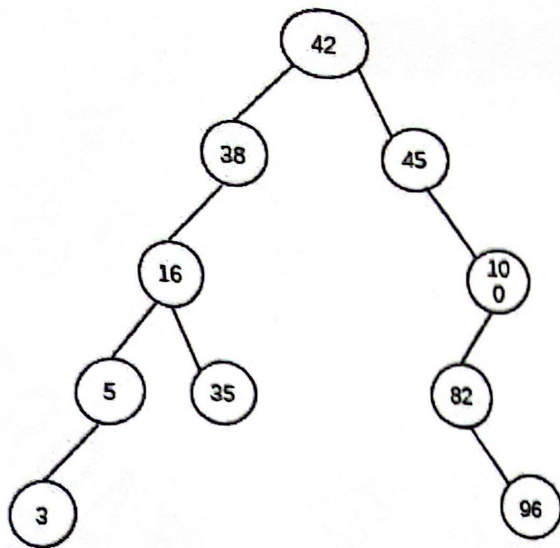


6. (1) 二叉排序树:



(2) 删除 50 后，删除方式有两种：直接前驱（左边最大）或者直接后继（右边最小）进行替换。





## 五、算法设计题

### 1. 算法填空

- (1)  $\&(L.elem[i-1])$
- (2)  $L.length-1$
- (3)  $++p$
- (4)  $*p$
- (5)  $L.length-1$

2. 可以采用头插法和尾插法，这里采用头插法：

```

LinkList List_HeadInsert(LinkList &L)
{
    LNode *s, int x;
    L=(LinkList)malloc(sizeof(LNode));
    L->next=null;
    scanf("%d",&x);
    while(x!=9999){
        s=(LNode*)malloc(sizeof(LNode));
        s->data=x;
        s->next=L->next;
        L->next=s;
        scanf("%d",&x);
    }
    return L;
}
  
```

3. 不是重点（可以跳过）



# 2012 年参考答案

## 一、选择题

1. C
2. A 注意审题，不具有的特点
3. A
4. B 考察栈的原则：先进后出
5. D  $(r-17+30)\%30=20$   $r=7$
6. B
7. C 常考点
8. C
9. B 最小生成树：n 个顶点，n-1 条边
10. D 两两组合进行比较

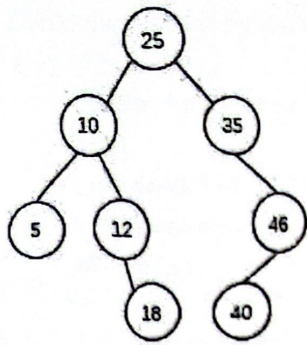
## 二、填空题

1.  $O(n^2)$
2. 栈 一般应用于：递归，函数，表达式求值，深度优先遍历
3.  $O(n)$  从头部遍历到尾部
4. 根 画一个草图
5.  $P->lchild!=null || p->rchild!=null$
6.  $2^{k-2}$  最少的情况是：最后一层只有一个结点，即  $2^{k-1-1}-1+1$
7. 回路或者环
8. 2 在 m 阶的 B 树中，  
①根结点中的关键字的个数为 n，满足： $1 \leq n \leq m-1$ ；  
②除开根结点以外，其他结点，满足： $[m/2]-1 \leq n \leq m-1$   
(为向上取整符号)
- 在 m 阶的 B<sub>+</sub> 树中，  
①根结点中的关键字的个数为 n，满足： $1 \leq n \leq m$ ；  
②除开根结点以外，其他结点，满足： $[m/2] \leq n \leq m$   
(为向上取整符号)
9. 17 画出对应的小根堆
10. n 当 n 个值成有序排列时，即是单支树，高度也为 n

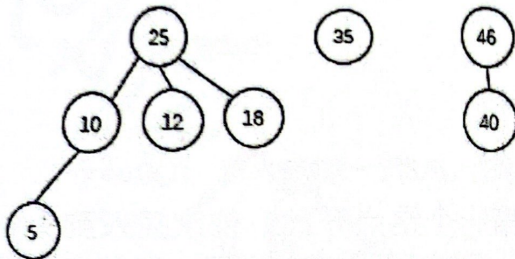
## 三、按照要求回答下列问题

21. (1)





(2)



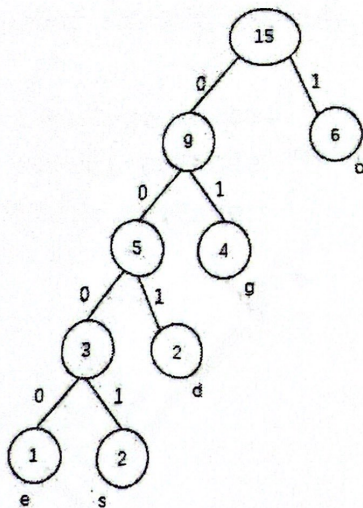
22. (1) 分析:

g:4 o:6 e:1 s:2 d:2

新建  $n-1$  个结点

总结点数  $2n-1 = 2*5-1=9$

哈夫曼树:



(2) o:1 g:01 d:001 s:0001 e:0000

Wpl:  $6*1+4*2+2*3+(1+2)*4=32$

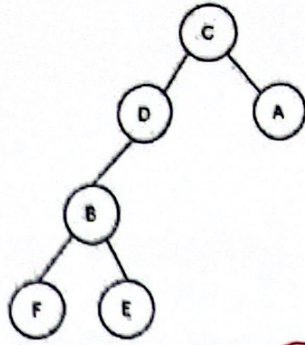
(3) 0110110000001011100101110010001

$4*2+6*1+1*4+2*4+2*3=32$

23. (1) 广度优先遍历: CDABFE

对应生成树:





(2) 题目应为 C 点到各点的最短路径:

C->A:7 C->D:8 C->B:18 C->F:27 C->E:33

#### 四、程序设计题

1.  $i < j \&\& r[i].key < x.key$

$r[i] = x$

2.

(1) 基本思想: 创建一个头结点, 然后从第一个结点开始, 依次插入到头结点的后面(头插法), 直到最后一个结点为止, 则实现了链表逆置, 尾结点变成头结点, 然后删除头结点即可。

(2)

```

typedef struct LNode
{
    ElemType data;
    struct LNode *next;
    LNode, *LinkList;
}
  
```

(3)



```

LinkList Reverse(LinkList &L)
{
    LNode *p,*r,*q; //q用来删除头结点
    p=h;
    L=(LinkList)malloc(sizeof(LNode));
    L->next=null;
    while(p)
    {
        r=p->next;
        p->next=L->next;
        L->next=p;
        p=r;
    } //已经倒置完毕
    q=L;
    L=q->next;
    free(q); //将L指向第一个结点，进而删除结点
    return L;
}

```

- 3.
- (1) 基本思想：在对二叉树进行中序遍历过程中，将其加入到栈（先进后出，倒置思想）中，最后依次出栈。
- (2)

```

typedef struct BiTNode
{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiLNode,*BiTree;

```

- (3)



---

```
void Inorder(BiTree root)
{
    if(root!=null)
    {
        Inorder(T->lchild);
        visit(T);
        S.data[++S.top]=T->data;//将元素入栈
        Inorder(T->rchild);
    }
    while(!stackEmpty(S))
    {
        x=S.data[S.top--];//依次出栈
        printf("%d",x);
    }
}
```